MANUALE SVILUPPATORE – PIATTAFORMA SKILLSHARE

Corso di Ingegneria del Software – A.A. 2025/2026

Gruppo di lavoro: Emanuele Beccari (0001173921), Ludovica Govoni (0001162689), Victoria La Commare (0001161558), Samuele Pederiva (0001173829)

---

1. Introduzione

Il presente **Manuale dello Sviluppatore** fornisce le linee guida tecniche e operative per l'installazione, la configurazione, la comprensione architetturale e la manutenzione della piattaforma web Skillshare. Il documento è rivolto a collaboratori e sviluppatori che intendono contribuire al progetto o analizzare le scelte implementative adottate dal team.

Skillshare è un'applicazione web P2P ideata per favorire lo scambio reciproco di competenze ed esperienze tra utenti, dalla programmazione alle lingue, fino alle attività pratiche. La piattaforma non prevede transazioni monetarie: ciascun iscritto riveste contemporaneamente il ruolo di docente e discente. Tra le funzionalità chiave figurano la gestione dei profili e annunci, la ricerca avanzata con filtri, la messaggistica interna, la gestione delle prenotazioni e un sistema di recensioni e reputazione.

L'applicazione è sviluppata interamente in **Java** sfruttando il framework **GWT (Google Web Toolkit)**, che suddivide l'architettura in quattro moduli principali:

- **skillshare-client**: contiene il codice dell'interfaccia utente (front-end) compilato in JavaScript.
- **skillshare-server**: gestisce la logica di business, l'erogazione delle **Servlet RPC** e la persistenza.
- **skillshare-shared**: raccoglie le interfacce dei servizi RPC e i **DTO (Data Transfer Object)** condivisi tra client e server.
- **skillshareIT**: esegue tramite **Selenium** i test di accettazione che simulano le interazioni dell'utente sull'interfaccia grafica (registrazione, login, annunci, chat, recensioni).

Per la gestione dello stato e la memorizzazione dei dati viene impiegato **MapDB**. L'intero ambiente di esecuzione è isolato tramite **Docker** per garantire l'avvio immediato dell'applicazione su qualsiasi macchina host.

Per garantire la separazione delle responsabilità tra la logica applicativa e la presentazione estetica, gli stili sono stati esternalizzati in fogli di stile **CSS** dedicati. Ad ogni elemento interattivo della GUI sono stati assegnati **ID HTML** e **classi CSS** univoche per permettere un ancoraggio solido all'interno delle classi **Page Object** della suite di test.

---

2. Gestione della persistenza

Il livello di persistenza dei dati è stato ingegnerizzato per rispondere a due esigenze fondamentali: garantire la thread-safety in un ambiente web multi-thread e supportare in modo dinamico sia l'esecuzione su Docker sia la fase di test automatizzati con Maven. Il sistema è suddiviso in due componenti architetturali principali, ossia le classi *DatabaseCore* e *DatabaseUtente*.

2.1 DatabaseCore.java

La classe *DatabaseCore.java* inizializza e gestisce il ciclo di vita del database. Poiché le Servlet RPC possono ricevere richieste concorrenti, tutti i metodi di accesso e modifica dello stato sono dichiarati come *synchronized*. Questo previene race condition sull'istanza del database.

- ***inizializzaDB()***. Serve per configurare la classe ed è richiamato automaticamente al primo caricamento tramite un blocco static. Ha il compito di istanziare l'oggetto *db* applicando la configurazione corretta in base al contesto di esecuzione. Se il flag *testMode* è attivo, istanzia un database volatile in memoria (*DBMaker.memoryDB()*). In ambiente di produzione/Docker, verifica la presenza della variabile *DATA_DIR* per instradare il salvataggio del file *progetto_sweng.db* nel volume persistente. Configura parametri di sicurezza fondamentali: abilita le transazioni (*.transactionEnable()*), disabilita i lock di sistema sui file (*.fileLockDisable()*) per evitare conflitti di accesso, e si assicura che le risorse del database vengano rilasciate in automatico nel momento in cui il processo della JVM viene terminato (*.closeOnJvmShutdown()*).

- ***getDB()***. Funge da punto di accesso globale per ottenere l'istanza attiva del database. Prima di restituire l'oggetto *db*, verifica che non sia nullo o chiuso. Se lo è, invoca internamente *inizializzaDB()* per ripristinare la connessione in modo trasparente per i chiamanti.

- ***enableTestMode(), disableTestMode().*** Permettono di alterare a runtime la modalità di archiviazione, switchando tra la persistenza su file e il database in RAM. Modificano il flag interno *testMode*, forzano la chiusura del database corrente chiamando *close()*, e invocano *inizializzaDB()* per istanziare un ambiente pulito. Sono utilizzati estensivamente dall'infrastruttura di test per garantire l'isolamento dei dati a ogni esecuzione.

- ***commit()***. MapDB non salva le modifiche in tempo reale per motivi di performance; questo metodo invoca internamente *db.commit()* per scrivere in modo permanente su disco (o in RAM) le alterazioni apportate alle collezioni.

- ***close()***. Gestisce lo spegnimento controllato del database. Verifica l'esistenza e lo stato dell'istanza prima di rilasciare le risorse. Un blocco try-catch intercetta e silenzia eventuali eccezioni di chiusura per evitare la propagazione di errori bloccanti, resettando infine il riferimento *db* a null in modo che una successiva chiamata a *getDB()* possa re-inizializzare il sistema.

2.2 DatabaseUtente.java

La classe *DatabaseUtente.java* incapsula tutta la logica di lettura e scrittura sulle mappe concorrenti di MapDB, nascondendo la complessità del database ai livelli superiori dell'applicazione.

I metodi di questa classe gestiscono le operazioni CRUD relative agli utenti, garantendo l'integrità dei dati e adottando un sistema di caching interno volto a prevenire errori e conflitti durante l'esecuzione dei test.

- ***getUtenti(), getIndiceUsername().*** Sono i metodi fondamentali per il recupero delle collezioni *ConcurrentMap* dal database. Non si limitano a restituire le variabili, ma implementano un sistema di caching. Con *DatabaseCore.getDB()*, recuperano l'instanza attuale del DB e la confrontano con quella salvata in memoria. Se le istanze differiscono (evento tipico durante l'esecuzione ravvicinata dei test Maven, che riavvia il DB in testMode), o se il DB è chiuso, i metodi procedono a riaprire le mappe ("utenti" e "indice_username") sul nuovo database e aggiornano i riferimenti in cache. Infine, definiscono i serializzatori di MapDB (*Serializer.STRING*, *Serializer.JAVA*) al momento della creazione delle mappe.

- ***resetPerTest()***. Resetta forzatamente i riferimenti interni a null, costringendo i getter a ricaricare le strutture dati dal database alla successiva invocazione.
- ***registraUtente(Utente utente)***. Gestisce l'inserimento di un nuovo utente garantendo la consistenza dei dati. Il metodo verifica preliminarmente che la mail (chiave primaria) e l'username (chiave secondaria) non siano già presenti nelle rispettive mappe, sollevando un'eccezione in caso di conflitto. Poi, inserisce l'oggetto Utente nella mappa principale e salva la mappatura *Username -> Mail* nell'indice secondario. Infine, *DatabaseCore.commit()* per consolidare contemporaneamente entrambe le scritture su disco.
- ***getUtente(String mail)*.** Effettua un accesso diretto alla mappa principale in tempo costante O(1) utilizzando la mail come chiave.
- ***getUtenteByUsername(String username)***. Interroga prima la mappa *indice_username* per ottenere la mail associata allo username fornito, e successivamente invoca *getUtente(mail)* per recuperare l'oggetto completo, mantenendo prestazioni ottimali senza necessità di scorrere linearmente tutti i record.
- ***login(String mail, String password)***. Si occupa dell'autenticazione. Recupera l'utente tramite la mail e verifica la corrispondenza esatta della password. Restituisce l'oggetto *Utente* in caso di successo, altrimenti *null* per indicare il fallimento delle credenziali, proteggendo il backend da accessi non autorizzati.
- ***aggiornaUtente(Utente utenteModificato).*** Permette la modifica dei dati di un profilo esistente. Verifica l'esistenza della mail nella mappa e sovrascrive il record con l'oggetto aggiornato, invocando infine *DatabaseCore.commit()* per persistere la modifica.

---

3. Guida all'installazione e avvio

Per poter installare, compilare ed eseguire il progetto *Skillshare* su una nuova macchina a partire dal codice sorgente, è necessario che l'ambiente di sviluppo locale sia configurato con i seguenti strumenti software:

- **Java JDK 17+.** Il Java Development Kit (versione 17 o superiore) è il requisito base fondamentale per poter compilare il codice dell'applicazione GWT e avviare il server backend. Le istruzioni dettagliate per l'installazione sono riportate all'interno del file app/README.md fornito con il codice sorgente.
- **Apache Maven.** Strumento essenziale per la gestione delle dipendenze. Viene utilizzato per gestire l'intero ciclo di vita dell'applicazione: installazione delle librerie, compilazione del frontend e del backend, esecuzione dei test e pacchettizzazione.
- **Docker e Docker Desktop.** Fondamentale per garantire l'esecuzione standardizzata del software su qualsiasi sistema operativo. Docker Desktop fornisce inoltre una comoda interfaccia grafica per gestire e monitorare i container attivi, ed è scaricabile dal sito ufficiale di Docker.
- **Git.** Necessario per poter clonare il repository remoto del progetto sulla propria macchina locale e ottenere i file sorgenti.

Di seguito sono riportati tutti i passaggi per installare l'applicazione.

1. Installare Docker sul proprio dispositivo dal link *https://www.docker.com/products/docker-desktop/*.

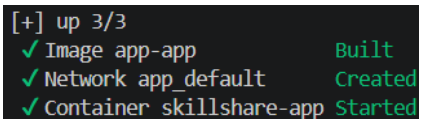
2. Creare su desktop una cartella vuota e aprirla dal proprio IDE.
3. Da terminale, lanciare il comando

   git clone https://github.com/VictoriaLaCommare/template_progetto_skillshare.git
4. Entrare nella cartella *'app'*. Per farlo, lanciare da terminale

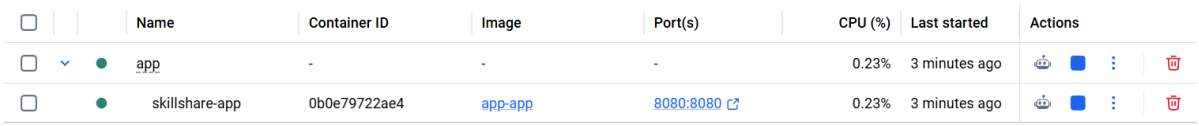
   cd template_progetto_skillshare

   e poi

   cd app
5. Nella cartella '*app*', lanciare *docker compose up -d --build* per iniziare l'installazione. Prima di lanciare questo comando, assicurarsi che Docker Desktop sia aperto e in esecuzione. L'installazione durerà qualche minuto.
6. Se il download è andato a buon fine, nel terminale si visualizzerà

   Dentro Docker, invece, la situazione sarà questa:

7. Non resta altro che lanciare l'applicazione. Per farlo, si può cliccare direttamente il link *8080:8080* all'interno di Docker, oppure digitare *http://localhost:8080/* all'interno della barra di ricerca del browser.

Per testare subito l'applicazione senza registrare un nuovo profilo, è possibile utilizzare una delle seguenti credenziali già presenti nel database:

Giulia Rossi:

mail: giuliarossi@gmail.com

password: giuliarossi

Gabriele Santini:

mail: gabrielesantini@gmail.com

password: gabrielesantini

Luca Argenti:

mail: lucaargenti@gmail.com

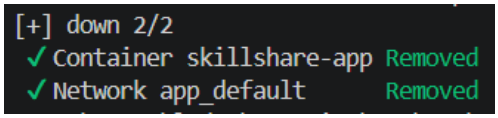
password: lucaargenti

Sara Fioca:

mail: sarafioca@gmail.com

password: sarafioca

Qualora si volesse disinstallare il programma, basterà cliccare l'icona del bidoncino rosso in Docker Desktop. In alternativa, lanciare da terminale *docker compose down*.



4. Guida generale ai Test

Il progetto ***Skillshare*** implementa una **suite di test automatizzati** volta a validare la stabilità e la correttezza del codice sorgente. L'architettura di testing è strutturata su due livelli complementari: i **test unitari**, mirati a validare il backend, e i **test di accettazione**, progettati per il collaudo dell'interfaccia grafica.

Per prevenire l'alterazione dei dati persistenti, ogni esecuzione della test suite innesca automaticamente il database in memoria tramite la proprietà **testMode** (già trattata). Questa soluzione garantisce un **isolamento totale** dell'ambiente di prova, preservando l'integrità del file di database destinato all'uso reale.

4.1 Test Unitari (JUnit)

I **test unitari** verificano le singole componenti software del backend. Per la loro stesura è stato adottato **JUnit 5**.

Le *best practice* del testing unitario impongono che i test siano veloci, ripetibili e, soprattutto, che non dipendano da stati esterni condivisi come file system o database fisici, che rallenterebbero l'eseguibilità. Per rispettare questo principio senza dover configurare complessi strati di simulazione, il sistema attiva automaticamente la proprietà **testMode** (descritta nel capitolo precedente). Questo espediente instrada la persistenza su un database interamente in **RAM**, garantendo l'isolamento e preservando il file di produzione.

- **Comando di esecuzione**: **mvn test** Il comando esegue una verifica logica rapida sul codice backend (classi ***Test.java**), mentre l'interfaccia **GWT** non viene compilata e non viene avviato alcun server.

4.2 Test di Integrazione (Selenium)

I **test di accettazione** verificano l'intera applicazione nel suo complesso, simulando il comportamento di un utente reale che interagisce con l'interfaccia grafica all'interno di un browser. Questi test sono implementati con **Selenium WebDriver** e sono isolati all'interno del modulo dedicato **skillshare-it**, utilizzando il suffisso ***IT.java**.

- **Comando di esecuzione**: **mvn verify**

Invocando questa fase del ciclo di vita di Maven, il sistema esegue un collaudo completo, che include automaticamente anche il passaggio precedente di mvn test. Successivamente, Maven compila il codice GWT in JavaScript, pacchettizza l'applicazione, avvia un server web locale temporaneo (tramite il plugin di Jetty) ed esegue i test Selenium. Al termine della suite, il server Jetty viene spento automaticamente.

Accortezza operativa

Sia l'ambiente Docker che quello di test automatizzato sono configurati nativamente per esporre l'applicazione sulla **porta 8080** del computer locale. Poiché due servizi differenti non possono restare in ascolto contemporaneamente sulla stessa porta di rete, è fondamentale assicurarsi che l'applicazione su Docker non sia in esecuzione quando si lancia la suite di test.

Se si tenta di lanciare mvn verify mentre il container Docker è attivo, i test di integrazione falliranno, in quanto il server Jetty non riuscirà ad avviarsi. Prima di lanciare i test automatizzati che coinvolgono l'interfaccia grafica, è obbligatorio spegnere i container tramite il comando da terminale:

- **docker compose down**

5. Implementazione dei Casi d'Uso

Nota: Componente comune di navigazione (AppNavbar)

Per garantire un'esperienza utente coerente, fluida e reattiva su tutte le schermate della web app, l'architettura frontend di Skillshare si avvale di un componente di navigazione globale centralizzato denominato **AppNavbar**.

- **Ruolo architetturale e Responsive State:** La classe AppNavbar estende Composite ed è progettata per adattarsi dinamicamente allo stato di autenticazione dell'utente. All'istanziazione, riceve come precondizione l'entità Utente attualmente loggato (o null se la sessione non è attiva). Sulla base di questo stato, la barra di navigazione compone programmaticamente i widget di GWT (Button, Anchor, HorizontalPanel), mostrando o nascondendo sezioni protette quali la Bacheca, la Ricerca, "I Miei Annunci", "Le Mie Richieste", la Chat, la Wishlist e il Profilo.
- **Iniezione e Navigazione Non Bloccante:** Il componente viene istanziato e iniettato dinamicamente all'interno di ciascuna vista principale (es. HomePageGUI, BachecaProposteGUI, CreaWishlistGUI, ecc.), passando l'istanza dell'utente corrente per mantenere la continuità della sessione durante i cambi di pagina gestiti client-side.
- **Standardizzazione degli ID per il Testing E2E:** Per agevolare l'automazione dei test di accettazione end-to-end con Selenium WebDriver, a ciascun pulsante interattivo della navbar è stato associato uno specifico ID HTML univoco (come #nav-btn-home, #nav-btn-bacheca, #nav-btn-chat, #nav-btn-wishlist, #nav-btn-profilo e #nav-btn-logout). Questo consente ai Page Object (come HomePage o WishlistPage) di individuare deterministicamente i controlli di navigazione ed eseguire i test di transizione delle viste in modo robusto e indipendente dai layout CSS.

5.1 Autenticazione e accesso (UC01 e UC02)

I casi d'uso "Registrazione Utente" (UC01) e "Login Utente" (UC02) presidiano l'ingresso nella piattaforma: permettono ai nuovi visitatori di creare un account validando il formato dei dati e garantendo l'unicità di email e username, e consentono agli iscritti di autenticarsi in modo sicuro per sbloccare le funzionalità riservate alla community.

Skillshare-shared

- **Entità di Dominio (Utente):** classe serializzabile (java.io.Serializable) che rappresenta l'iscritto incapsulando nome, cognome, username, email, data di nascita, password (sottoposta a hashing sul server), biografia, URL foto profilo e tag di competenza.
- **Interfacce Servizi e Proxy (GestoreAutenticazione, Registrazione, RegistrazioneAsynch):**

  definiscono il contratto di comunicazione RPC (@RemoteServiceRelativePath("registrazione")) per i metodi di creazione account (registraUtente) e verifica credenziali (login / effettuaLogin), sia in versione sincrona che asincrona (AsyncCallback).
- **Validazione centralizzata (FieldVerifier):** collocata nel modulo shared per poter essere condivisa senza duplicazioni sia dal client GWT che dal server (AutenticatoreProxy), espone i metodi statici per verificare i vincoli formali su nome e cognome (minimo 3 lettere), username (da 3 a 25 caratteri, almeno 3 lettere), email (formato valido con @ e .), data di nascita non futura e password (minimo 8 caratteri).

Skillshare-client

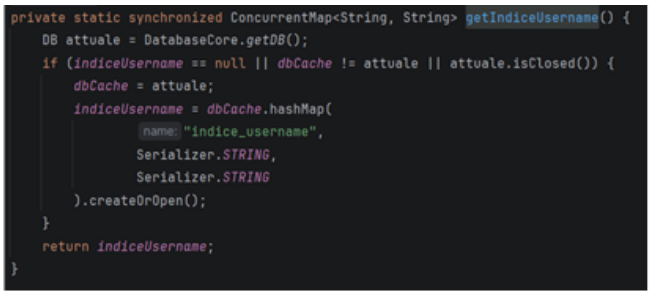
La classe HomePageGUI gestisce la schermata iniziale di benvenuto (che mostra anche l'anteprima degli "Ultimi Annunci Pubblici" tramite ottieniTuttiGliAnnunci) e commuta dinamicamente la vista verso i moduli di registrazione (mostraFormRegistrazione) e di login (mostraFormLogin):

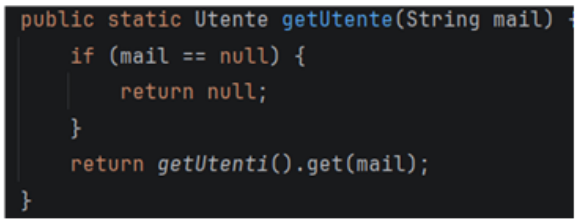
- **Flusso di Registrazione (UC01):** prima di inviare la richiesta al server, il client valida ciascun campo tramite FieldVerifier. Se uno o più campi non rispettano i criteri, l'invio viene bloccato, i bordi dei campi errati vengono evidenziati in rosso (.input-errore) e vengono mostrati i suggerimenti puntuali accanto agli input. Se la validazione è superata, parte la chiamata RPC asincrona: in caso di successo compare il messaggio di conferma verde e l'utente viene reindirizzato automaticamente alla schermata di Login.
- **Flusso di Login (UC02):** l'utente inserisce email e password e preme "Accedi". Al ritorno positivo della callback con l'oggetto Utente autenticato, la GUI istanzia la barra di navigazione superiore (AppNavbar), sbloccando la navigazione verso tutte le viste applicative.

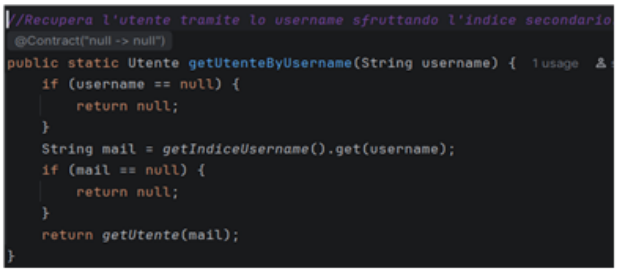
Skillshare-server

Sul backend, l'architettura di autenticazione combina la servlet RegistrazioneImp, il pattern **Proxy**(AutenticatoreProxy e AutenticatoreReale), il gestore di crittografia PasswordManager e il layer di persistenza DatabaseUtente:





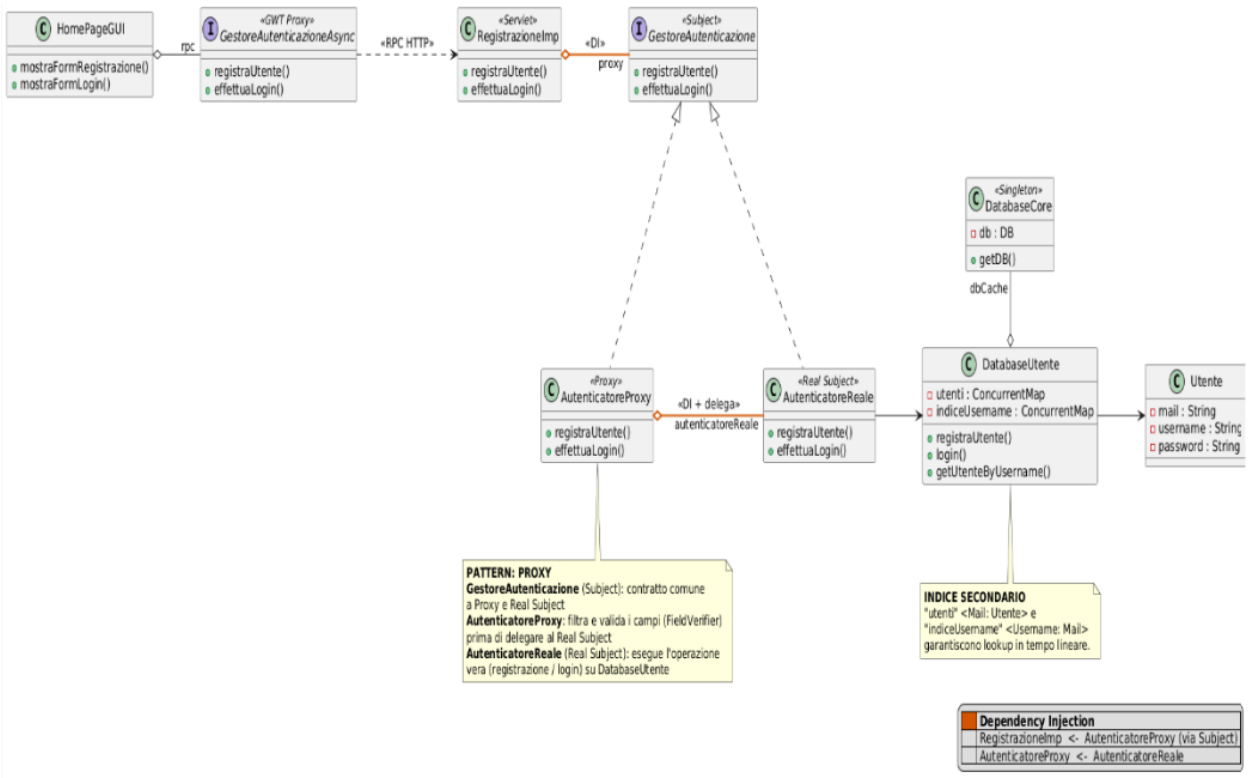




- **Pattern Proxy (GestoreAutenticazione, AutenticatoreProxy, AutenticatoreReale).**

  La servlet RegistrazioneImp non accede direttamente al database, ma inietta e delega le richieste ad AutenticatoreProxy.

  - AutenticatoreProxy (Proxy): intercetta le richieste provenienti dal client e riesegue lato server tutti i controlli formali tramite FieldVerifier e i controlli di non-nullità, sollevando eccezioni o bloccando l'operazione in caso di input anomali.
  - AutenticatoreReale (Real Subject): viene invocato dal Proxy solo a validazione superata ed esegue l'operazione effettiva su DatabaseUtente, rimanendo snello, isolato e aderente al principio **SRP** (Single Responsibility Principle).



- **Sicurezza delle Password (PasswordManager):** le password non vengono salvate in chiaro, ma sottoposte a hashing sicuro prima della memorizzazione e del confronto in fase di login.
- **Correzioni evolutive: Indice Secondario MapDB (indice_username) e Caching per i Test.** Durante lo sviluppo iniziale, DatabaseUtente memorizzava gli utenti in una sola mappa utenti avente la mail come chiave primaria. Tuttavia, poiché altri moduli (GestoreAnnunciImpl, GestoreLeveling, GestoreRichiesteServiceImpl) identificano gli utenti tramite username, la mancanza di un vincolo di unicità sullo username poteva causare collisioni (es. due utenti con lo stesso username associati alle stesse statistiche). Il problema è stato risolto introducendo una seconda ConcurrentMap<String, String> su MapDB denominata indice_username (che mappa username -> mail): in fase di registrazione viene verificata l'unicità di entrambi i campi, e il metodo getUtenteByUsername(username) permette di risalire alla mail e all'oggetto Utente in tempo costante $O(1)$.Inoltre, DatabaseUtente mantiene un riferimento dbCache all'istanza corrente di DatabaseCore.getDB(): quando i test Maven aprono e chiudono ripetutamente il DB in memoria (enableTestMode() / disableTestMode()), DatabaseUtente rileva il cambio di istanza e riapre automaticamente le mappe, prevenendo NullPointerException o riferimenti a database chiusi.

SkillshareIT

- **Test Unitari (AutenticatoreProxyTest, RegistrazioneImpTest, DatabaseUtenteIndiceTest):** validano in isolamento (con DB in RAM) il filtraggio degli input errati da parte di AutenticatoreProxy, il blocco di registrazioni con email o username duplicati, il funzionamento dell'indice secondario indice_username e la correttezza del login.

- **Test di Accettazione Selenium (RegistrazioneIT e LoginIT):** tramite il Page Object RegistrazionePage, simulano nel browser l'intero percorso di creazione di un nuovo profilo (verificando sia l'evidenziazione rossa dei campi in caso di email/password non valide, sia l'Happy Path di registrazione completata) e il successivo login con sblocco della AppNavbar.

5.2 Personalizzazione del profilo (UC03)

Il caso d'uso "Personalizza Profilo" rappresenta una delle funzionalità core del sistema, la quale permette agli utenti autenticati di arricchire la propria identità pubblica all'interno della piattaforma, inserendo una breve biografia, l'URL della propria foto profilo e i tag relativi alle competenze offerte.

Skillshare-shared

L'architettura espone i servizi di business tramite l'interfaccia sincrona ***GestoreProfilo***, la quale estende *RemoteService* e utilizza l'annotazione *@RemoteServiceRelativePath("profilo")* per mappare l'endpoint sul server.

Di pari passo, l'interfaccia ***GestoreProfiloAsync*** definisce i medesimi metodi implementando il pattern asincrono necessario al client per non bloccare l'esecuzione della GUI, restituendo i risultati attraverso oggetti *AsyncCallback*.

La classe ***FieldVerifier*** incapsula tutte le logiche di controllo dell'input, esponendo metodi specifici per il caso d'uso come *isValidBio* e *isValidTag*.

Skillshare-client

La classe ***PersonalizzaProfiloGui*** gestisce la schermata di modifica del profilo.

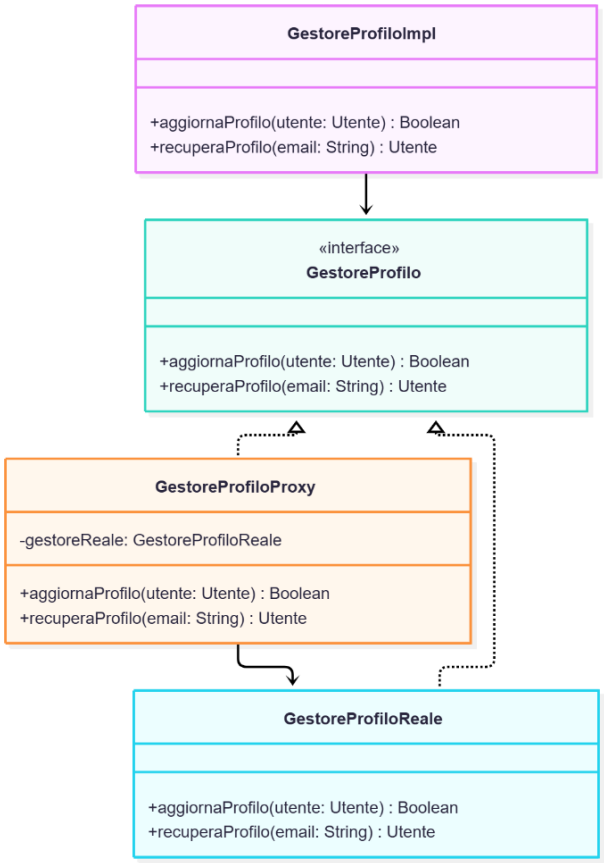
Il metodo *mostra(Utente utenteLoggato)* riceve in input l'entità utente corrente e costruisce il layout utilizzando i widget standard di GWT. I campi di input vengono pre-popolati con i dati attuali dell'utente, se presenti. È stata inoltre integrata la navigazione verso altre viste, istanziando dinamicamente la *AppNavbar* e il popup per la visualizzazione del profilo pubblico tramite ***ProfiloUtenteGUI***. Ad ogni elemento interattivo della form sono stati assegnati specifici ID HTML per facilitare l'ancoraggio durante la fase di testing.

Alla pressione del bottone "Salva modifiche", prima di instradare la richiesta verso il server, la classe applica le logiche di validazione richiamando i metodi statici del ***FieldVerifier***. In caso di esito negativo, l'invio viene bloccato e l'interfaccia si aggiorna istantaneamente, modificando le classi CSS per evidenziare i campi in errore e mostrando messaggi di allerta. Solo se la validazione lato client ha esito positivo, l'oggetto *utenteLoggato* viene aggiornato in memoria e trasmesso al backend tramite il proxy generato da *GWT.create(GestoreProfilo.class)*.

La comunicazione avviene in modo non bloccante attraverso l'uso di un *AsyncCallback<Utente>*. In caso di successo, l'interfaccia restituisce un feedback visivo positivo all'utente ripristinando il normale stato dei campi; in caso di errore di rete o validazione lato server, viene mostrato il dettaglio dell'eccezione lanciata dal backend.

Skillshare-server

La classe ***GestoreProfiloImpl*** estende *RemoteServiceServlet* e implementa l'interfaccia condivisa ***GestoreProfilo***, fungendo da punto di ingresso per le chiamate provenienti dal client. Invece di implementare direttamente la logica di business o l'accesso ai dati, la servlet delega le richieste a un oggetto proxy istanziato internamente. Questo approccio centralizza il controllo del flusso prima di qualsiasi operazione di lettura o scrittura.



- ***GestoreProfiloProxy*** = oggetto surrogato.

  Implementa l'interfaccia ***GestoreProfilo*** e intercetta le chiamate prima che raggiungano la logica di business. Nel metodo *aggiornaProfilo*, esegue una validazione lato server riutilizzando i metodi di ***FieldVerifier*** (es. *isValidBio*, *isValidTag*). Se i parametri risultano assenti, nulli o violano i vincoli di business, solleva un' *IllegalArgumentException* e blocca l'esecuzione. Inoltra la richiesta a *GestoreProfiloReale* solo se tutti i controlli sono superati, garantendo il rispetto del *Single Responsibility Principle*.

- ***GestoreProfiloReale*** = real subject.

  Viene invocato dal Proxy solo dopo il superamento dei controlli ed è l'unico componente responsabile dell'interazione diretta con *DatabaseCore*. Nel metodo *aggiornaProfilo*, gestisce la struttura dati HTreeMap per memorizzare l'oggetto *Utente* serializzato (utilizzando l'email come chiave primaria) e aggiorna la mappa *indice_username*. Durante il recupero del profilo, implementa un meccanismo di sicurezza restituendo una nuova istanza dell'oggetto Utente contenente i soli dati pubblici ed escludendo la password.

- ***GestoreProfilo*** =subject (interfaccia astratta del servizio)

  Definisce il contratto astratto dei metodi di business (*aggiornaProfilo*, *recuperaProfilo*) condiviso dal Proxy e dal soggetto reale, consentendo al chiamante di interagire con il servizio in modo totalmente trasparente.

- ***GestoreProfiloImpl*** = client.

  Estende *RemoteServiceServlet* e costituisce il punto d'ingresso per le richieste provenienti dal front-end. Invece di eseguire direttamente l'accesso ai dati, delega l'esecuzione delle operazioni all'interfaccia *GestoreProfilo*, che fa capo al *GestoreProfiloProxy*.

Il layer server è ampiamente coperto da test unitari implementati con JUnit 5 e Mockito. I test istanziano la servlet simulando il container web e l'oggetto *HttpServletRequest* per verificare il corretto funzionamento dei metodi senza un vero server in esecuzione. Casi di test specifici verificano il successo dell'aggiornamento e la sicurezza dei dati restituiti, assicurando tramite un'asserzione *assertNull* che la password non sia mai presente nel profilo recuperato. Ulteriori test verificano le logiche di filtraggio.

SkillshareIT

La classe ***PersonalizzaProfiloIT*** gestisce il ciclo di vita delle sessioni di test istanziando un ChromeDriver.

- ***@BeforeEach*** prepara l'ambiente di test inizializzando la pagina e avviando la navigazione verso l'URL base dell'applicazione.
- ***@AfterEach*** garantisce l'indipendenza delle esecuzioni. Al termine di ogni test il sistema cerca e preme automaticamente il pulsante di logout (*nav-btn-logout*) se presente, resettando la sessione utente.

Per evitare di accoppiare la logica di test con la struttura HTML, l'interazione con il DOM è stata astratta nella classe **PersonalizzaProfiloPage**.

- Attraverso l'uso di ***PageFactory.initElements*** e delle annotazioni ***@FindBy***, i widget dell'interfaccia sono mappati in modo dichiarativo tramite i rispettivi ID (*prof-bio*, *prof-foto*, *prof-tag*, *prof-submit*, *prof-message*).
- Le interazioni sono esposte tramite metodi ad alto livello (come *compila* e *salva*). Questi metodi implementano le *Explicit Waits* tramite *WebDriverWait*, permettendo al test di attendere dinamicamente i tempi di caricamento asincroni di GWT senza ricorrere a pause statiche bloccanti.

Casi di test implementati:

- ***datiValidi_aggiornamentoCompletatoConSuccesso***. Compila il form con dati corretti (biografia, foto, tag) e innesca il salvataggio. Un'asserzione finale verifica che il sistema confermi l'operazione mostrando a schermo l'etichetta di risultato con la parola "successo".
- ***bioNonValida_evidenziaErrore***. Verifica i limiti di validazione del client inserendo una stringa anomala (1000 caratteri) nel campo biografia. L'asserzione conferma che il sistema blocchi tempestivamente l'inoltro della richiesta e notifica l'utente con un messaggio di allerta visibile (es. "controlla gli errori" o "non è valida").

5.3 Creazione e gestione annunci (UC04 e UC05)

Il caso d'uso dedicato alla creazione e alla gestione degli annunci rappresenta il fulcro delle interazioni all'interno della bacheca dell'applicazione. L'architettura di questo modulo è stata progettata per gestire non solo la creazione di nuove offerte di scambio, ma l'intero ciclo di vita dell'annuncio (pubblicazione, modifica, eliminazione).

Skillshare-shared

L'entità centrale è rappresentata dalla classe ***Annuncio***. Questa classe modella tutte le informazioni necessarie per uno scambio di competenze:

- *id*: identificativo univoco
- *titolo*, *descrizione*: dettagli testuali
- *competenzaOfferta*, *competenzaRichiesta*: categorizzazioni
- *autoreUsername*: riferimento all'utente creatore.

La classe implementa l'interfaccia *java.io.Serializable* per garantire che gli oggetti possano essere correttamente serializzati dal framework GWT e trasmessi sulla rete tra il client e il server.

L'interfaccia ***GestoreAnnunci*** estende *RemoteService* e definisce l'endpoint server tramite l'annotazione *@RemoteServiceRelativePath("annunci")*. Essa espone la firma di tutti i metodi di business necessari:

- *pubblicaAnnuncio*
- *recupero* (totale, per utente, preferiti, ricerca avanzata)
- *modificaAnnuncio*
- eliminaAnnuncio.

GestoreAnnunciAsync, invece, mappa i medesimi metodi aggiungendo il parametro *AsyncCallback*, essenziale per consentire al client di effettuare chiamate non bloccanti e mantenere l'interfaccia reattiva durante le latenze di rete.

Skillshare-client

Il livello di presentazione lato client affida la gestione del ciclo di vita degli annunci a due componenti principali, progettati per garantire un'interfaccia reattiva e un'esperienza utente fluida: ***PubblicaAnnunciGui*** e ***MieiAnnunciPanel*.**

PubblicaAnnunciGui orchestra la creazione di nuove inserzioni assemblando widget all'interno del metodo *mostra(Utente utenteLoggato)*. Prima di inoltrare qualsiasi richiesta di rete, il client esegue una validazione preventiva per assicurarsi che i campi non siano vuoti o nulli. In caso di anomalia, l'interfaccia reagisce istantaneamente evidenziando i campi critici con l'applicazione della classe CSS *input-errore* e mostrando messaggi di allerta mirati. Se i controlli superano la verifica, viene istanziato un oggetto *Annuncio* associato allo username dell'autore e trasmesso al backend tramite *GestoreAnnunciAsync*. Al successo della callback (*onSuccess*), il form viene ripulito e la vista viene aggiornata dinamicamente tramite il metodo *caricaAnnunci*.

MieiAnnunciPanel consente agli utenti autenticati di amministrare esclusivamente i post da loro creati. All'apertura, il sistema invoca in modo asincrono il metodo *ottieniAnnunciUtente* per recuperare la lista filtrata delle inserzioni. Ciascun annuncio viene visualizzato sotto forma di card, dotata di pulsanti interattivi provvisti di ID univoci per agevolare il collaudo tramite test di accettazione automatizzati.

- La pressione del bottone ***Modifica*** apre una finestra modale nativa pre-popolata con i dati attuali del post. Dopo aver apportato le modifiche desiderate, la chiamata a *modificaAnnuncio* aggiorna il record sul server e ricarica la vista.
- Per prevenire cancellazioni accidentali, la richiesta di rimozione attiva un popup modale di conferma ("Sei sicuro di voler eliminare questo annuncio?"). Solo alla conferma esplicita (*btn-conferma-elimina-si*), il client invoca *eliminaAnnuncio* passando l'ID e lo username di controllo, provvedendo a rimuovere la card dalla schermata al completamento dell'operazione.

Skillshare-server

Il livello server per la gestione e pubblicazione degli annunci è implementato nella classe ***GestoreAnnunciImpl***, la quale estende *RemoteServiceServlet* e fornisce le implementazioni concrete delle chiamate RPC definite nel livello *shared*. Questo componente garantisce la persistenza su database MapDB e orchestra le regole di business.

L'accesso alla collezione degli annunci è standardizzato dal metodo *getMappaAnnunci()*, che crea o recupera una struttura dati di tipo *HTreeMap<String, Annuncio>* serializzata. Ad ogni operazione di scrittura, viene invocato il metodo *DatabaseCore.commit()* per consolidare fisicamente le modifiche sul file di database.

Prima di eseguire metodi che alterano lo stato del database (*modificaAnnuncio* ed *eliminaAnnuncio*), il sistema implementa un controllo di sicurezza. L'applicazione recupera l'annuncio preesistente (*annuncioSicuro*) e confronta il suo autore originale (*getAutoreUsername()*) con lo username dell'utente che ha inoltrato la richiesta. Se i valori non coincidono, l'operazione viene immediatamente bloccata restituendo *false*, impedendo così manipolazioni non autorizzate. In fase di modifica, inoltre, vengono aggiornati unicamente i campi testuali descrittivi, preservando l'immutabilità dell'ID e dell'autore.

Il metodo *cercaAnnunci* esegue controlli strutturati per abbinare le competenze offerte, le competenze richieste e una ricerca a testo libero all'interno del titolo o della descrizione (*matchKeyword*).

Il metodo *ottieniAnnunciPreferiti* restituisce all'utente un feed personalizzato basato sugli autori preferiti o sui tag desiderati.

Per gestire in modo disaccoppiato e reattivo le dipendenze legate al ciclo di vita degli annunci, l'architettura implementa il design pattern Observer. Questo approccio garantisce che la cancellazione di un annuncio si propaghi in automatico ai moduli dipendenti (come il sistema delle richieste di scambio) preservando la coerenza del database, senza però creare un accoppiamento forte tra i servizi.

- ***GestoreAnnunciImpl (Subject)***

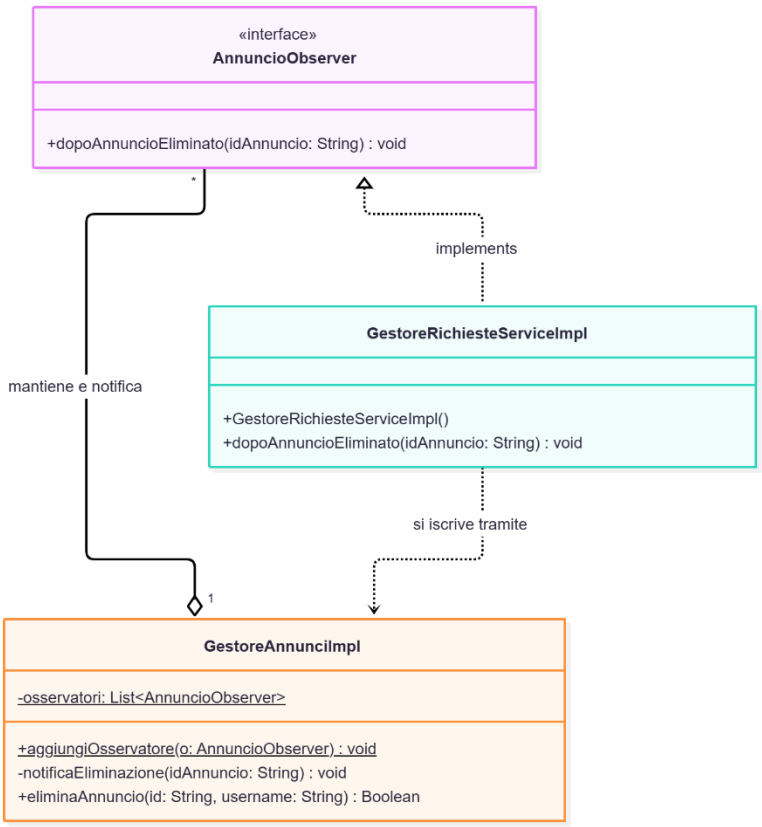
  Mantiene al suo interno una lista statica di osservatori registrati. Nel momento in cui un utente elimina un annuncio dal database, il Subject invoca il metodo privato *notificaEliminazione*, il quale itera sulla lista e innesca automaticamente il metodo di callback su ciascun iscritto.

- ***AnnuncioObserver (Observer)***

  Interfaccia che funge da contratto per tutti i componenti del sistema interessati a reagire agli eventi degli annunci. Definisce la firma del metodo *dopoAnnuncioEliminato(String idAnnuncio)*, fornendo un punto di ingresso standard per ricevere le notifiche.

- ***GestoreRichiesteServiceImpl (Concrete Observer)***

  Rappresenta l'osservatore concreto che reagisce materialmente all'evento. All'interno del proprio costruttore, la classe si iscrive autonomamente al Subject invocando *GestoreAnnunciImpl.aggiungiOsservatore(this)*. Quando riceve la notifica (tramite l'override del metodo *dopoAnnuncioEliminato*), il servizio interroga le HTreeMap di *DatabaseCore*: se individua richieste di scambio associate a quell'annuncio, ne forza lo stato impostandolo a *StatoRifiutata* e consolida le modifiche tramite *commit()*, invalidando così le negoziazioni rimaste in sospeso.



L'introduzione di questo design pattern introduce tre grandi vantaggi:

- **Loose coupling.** Il modulo che elimina l'annuncio non deve conoscere l'esistenza o l'implementazione interna degli altri moduli, ma si limita a notificare al sistema che l'azione è avvenuta.
- **Rispetto del principio open-closed.** Se in futuro si vorrà aggiungere una nuova funzionalità legata all'eliminazione, basterà creare una nuova classe che implementa *AnnuncioObserver*, senza dover toccare e rischiare di rompere il codice del gestore annunci esistente.
- **Rispetto del Single Responsability Principle.** L'utilizzo dell'interfaccia *AnnuncioObserver* garantisce che ogni componente del sistema mantenga un unico compito ben definito. Il gestore degli annunci ha l'esclusiva responsabilità di operare sul ciclo di vita dell'entità *Annuncio* (come, ad esempio, la sua rimozione dal database). Tutte le azioni derivate non inquinano il codice del gestore principale, ma vengono delegate ed eseguite in modo totalmente isolato dalle rispettive classi *Observer* in ascolto.

La robustezza della logica server è garantita dai test unitari della classe *GestoreAnnunciTest*. I test verificano il ciclo di vita completo dell'entità *Annuncio* (pubblicazione, recupero filtrato per utente, modifica sicura ed eliminazione definitiva). Le asserzioni verificano non solo i percorsi ideali, ma si assicurano anche che il sistema gestisca correttamente scenari con database vuoti senza sollevare eccezioni impreviste.

SkillshareIT

I test di accettazione sono implementati tramite Selenium WebDriver. I test collaudano l'intero ciclo di vita dell'annuncio, operando direttamente sull'interfaccia utente.

Poiché l'inserimento di un annuncio richiede necessariamente un utente autenticato, la configurazione del test (*@BeforeEach*) automatizza preliminarmente la creazione e il login di un utente fittizio. Solo a login avvenuto la bacheca diviene accessibile e il test procede all'inizializzazione della classe *GestoreAnnunciPage*.

Per evitare di inquinare il database ad ogni esecuzione e compromettere test successivi, i test popolano una lista temporanea (*annunciDaPulire*) con i titoli degli annunci creati; al termine di ogni esecuzione, il sistema tenta di eliminare ciclicamente tutti gli elementi tracciati in lista, per poi concludere con un logout forzato che resetta la sessione del browser.

La classe *QuandoIlFormEVuoto* e *QuandoUnSoloCampoEVuoto* verificano la responsività della UI quando l'utente immette dati errati o parziali. Assert mirati confermano che il sistema applichi immediatamente il feedback visivo (come il bordo rosso) senza propagare eccezioni lato server.

Nello scenario *QuandoIDatiSonoValidi*, il test compila tutti i campi richiesta. L'asserzione verifica sia l'avvenuto invio ("messaggio di successo") sia il reset automatico del form di inserimento.

La classe *GestioneAnnunci* testa le operazioni di aggiornamento e cancellazione. Il test *annuncioPubblicato_puoiModificarlo* simula il click sul bottone di modifica, popola dinamicamente il popup di edit e verifica, interrogando l'interfaccia, che il vecchio titolo sparisca a favore di quello nuovo aggiornato. Allo stesso modo, il test di eliminazione accerta che, cliccando l'apposito pulsante e confermando tramite popup, il nodo HTML associato alla *card* scompaia definitivamente dalla vista.

5.4 Visualizzazione e ricerca annunci (UC06 e UC07)

Il modulo di esplorazione e ricerca consente agli utenti di consultare l'intero catalogo delle proposte pubblicate nella community, filtrarle combinando più parametri di ricerca, ordinarle alfabeticamente o attivare un feed personalizzato ("Per Te") basato sui propri interessi.

Skillshare-shared

Le interfacce GestoreAnnunci e GestoreAnnunciAsync definiscono i contratti per l'interrogazione del catalogo:

- ottieniTuttiGliAnnunci(): restituisce la collezione completa degli annunci attivi.
- cercaAnnunci(String competenzaOfferta, String competenzaRichiesta, String keyword, String ordinamento): supporta il filtraggio multi-criterio e il sorting lato server.
- ottieniAnnunciPreferiti(String username): incrocia i dati dell'utente corrente per restituire una lista di annunci raccomandati.

Skillshare-client

La classe BachecaProposteGUI gestisce la visualizzazione del catalogo e l'interazione con i filtri. All'apertura della pagina, il client invoca ottieniTuttiGliAnnunci e popola dinamicamente una FlexTable(tabella-proposte) strutturata in 6 colonne: Titolo, Autore (cliccabile per ispezionare il profilo pubblico tramite ProfiloUtenteGUI), Descrizione, Competenza Offerta, Competenza Richiesta e Azione. Cliccando sul pulsante "🔍 Filtri Avanzati" (btn-filtri), viene visualizzato un PopupPanel modale (pannelloFiltri-bacheca) contenente:

- Tre campi TextBox per filtrare per competenza offerta (tB-Offerta), competenza richiesta (tB-Richiesta) e parola chiave libera (tB-Keyword).
- Una ListBox per selezionare il criterio di ordinamento (es. "Alfabetico per Titolo").
- Il pulsante **"Cerca"** (btn-esegui-ricerca), che raccoglie i valori inseriti e invoca cercaAnnunci, rigenerando la tabella con i soli risultati pertinenti.

- Il pulsante "❤️ **Per Te"** (btn-annunci-per-te), che chiude il popup e invoca ottieniAnnunciPreferiti, popolando la tabella con le proposte affini agli interessi dell'utente.

Skillshare-server

Nella classe GestoreAnnunciImpl, la logica di ricerca e raccomandazione è suddivisa in due metodi principali:

1. **Ricerca parametrica e Sorting (cercaAnnunci):** il metodo preleva tutti gli annunci e applica tre predicati booleani *case-insensitive* (matchOfferta, matchRichiesta, matchKeyword). Se un filtro è nullo o vuoto viene ignorato (restituendo true), altrimenti viene verificata la sottostringa tramite .toLowerCase().contains(...). Per la keyword, la ricerca viene effettuata sia sul titolo che sulla descrizione dell'annuncio. Gli annunci che soddisfano contemporaneamente tutti i filtri attivi (matchOfferta && matchRichiesta && matchKeyword) vengono aggiunti alla lista dei risultati. Infine, se il parametro ordinamento vale "titolo", la lista viene ordinata alfabeticamente tramite un comparatore compareToIgnoreCase.
2. **Feed personalizzato (ottieniAnnunciPreferiti):** sfrutta l'indice secondario DatabaseUtente.getUtenteByUsername(username) per risalire in tempo costante $O(1)$ alla mail dell'utente corrente. Con la mail, interroga due mappe distinte su MapDB: preferenze_match (per ottenere la lista profiliCuore degli utenti a cui è stato messo il "cuore") e wishlist_servizi (per estrarre i tag delle competenzeDesiderate dall'utente). Scansionando gli annunci (esclusi quelli pubblicati dall'utente stesso), il metodo seleziona tutti gli annunci in cui l'autore figura tra i preferiti oppure la competenza offerta contiene uno dei tag presenti nella Wishlist (autoreTraIPreferiti || competenzaInWishlist).

La classe GestoreAnnunciSearchTest verifica tramite test unitari JUnit il comportamento del motore di ricerca con filtri vuoti, con parole chiave e con ordinamento alfabetico.

SkillshareIT

Le classi BachecaPropositeIT e RicercaAvanzataIT (supportate dal Page Object BachecaPropostePage) testano l'apertura del pannello filtri, l'inserimento combinato dei parametri di ricerca, l'ordinamento dei risultati in tabella e il funzionamento dell'algoritmo "Per Te" in combinazione con i desideri espressi nella Wishlist.

5.5 Richiesta e gestione di uno scambio (UC08 e UC09)

I casi d'uso "Richiesta di Scambio" (UC08) e "Gestisci Richieste" (UC09) regolano l'intero flusso di negoziazione tra gli utenti della piattaforma. Il modulo consente a un utente autenticato di inviare formalmente una candidatura di scambio didattico all'autore di un annuncio in bacheca (prevenendo l'auto-scambio e gli invii duplicati) e permette a entrambe le parti coinvolte di monitorare le proposte attive e governarne le transizioni di stato: accettazione, rifiuto, annullamento, dichiarazione di terminazione e invio della recensione.

Skillshare-shared

Il modulo condiviso modella la transazione attraverso l'entità serializzabile (java.io.Serializable) RichiestaScambio, che raccoglie le informazioni fondamentali per instaurare e gestire il baratto di competenze:

- idRichiesta: identificativo univoco della proposta, generato tramite UUID.

- idAnnuncio: riferimento all'inserzione selezionata in bacheca.
- richiedente e ricevente: gli username rispettivamente dell'utente che propone lo scambio e del proprietario dell'annuncio.
- stato: oggetto di tipo StatoRichiesta che, alla creazione di una nuova proposta, viene inizializzato nello stato concreto StatoInSospeso e governa l'evoluzione della richiesta secondo il pattern **State**(StatoInSospeso, StatoAccettato, StatoInAttesaConferma, StatoCompletato, StatoRifiutata).
- Flag booleani dedicati al tracciamento della doppia conferma di terminazione e dell'invio delle recensioni.

La comunicazione RPC tra client e server è definita dalle interfacce GestoreRichiesteService (mappata sull'endpoint @RemoteServiceRelativePath("richieste")) e GestoreRichiesteServiceAsync, che espongono i metodi inviaRichiesta, ottieniTutteLeRichieste, ottieniRichiesteRicevute, accettaRichiesta, rifiutaRichiesta, dichiaraTerminazione e inviaRecensione.

Skillshare-client

Il livello di presentazione lato client suddivide l'interazione tra due viste principali: BachecaProposteGUI per l'invio della candidatura (UC08) e GestisciRichiesteGUI per l'amministrazione delle proposte inviate e ricevute (UC09).

1. Invio della proposta dalla Bacheca (BachecaProposteGUI): Durante il caricamento della tabella delle proposte (tabella-proposte), il client interroga preventivamente il backend tramite ottieniTutteLeRichieste(utenteLoggato.getUsername()) per conoscere le richieste già attive dell'utente corrente. Per ciascun annuncio mostrato nella FlexTable, la colonna "Azione" viene popolata dinamicamente applicando tre controlli logici:

- **Inibizione dell'auto-scambio:** se lo username dell'autore dell'annuncio coincide con quello dell'utente loggato (annuncio.getAutoreUsername().equals(utenteLoggato.getUsername())), il pulsante di richiesta viene disabilitato/omesso, impedendo all'utente di inviare proposte ai propri stessi annunci.
- **Prevenzione di richieste duplicate:** se nella lista delle richieste dell'utente figura già una richiesta associata a quell'idAnnuncio in stato "In sospeso" o "Accettata", l'interfaccia blocca ulteriori invii sostituendo il bottone con la dicitura di stato "In attesa di accettazione".
- **Inoltro della proposta:** se l'annuncio appartiene a terzi e non vi sono richieste pendenti, viene renderizzato il pulsante "Richiedi Scambio". Al click dell'utente, il client istanzia un oggetto RichiestaScambio popolandolo con l'idAnnuncio, lo username del richiedente (utente loggato) e quello del ricevente (autore del post), invocando in modo non bloccante inviaRichiesta. In caso di esito positivo (onSuccess), la GUI mostra una finestra modale di conferma ("Richiesta di scambio inviata con successo a [autore] per l'annuncio '[titolo]'.") e aggiorna istantaneamente la riga della tabella mostrando lo stato "In attesa di accettazione".

2. Gestione delle richieste (GestisciRichiesteGUI): La classe costruisce la vista "Le Mie Richieste" invocando all'avvio ottieniTutteLeRichieste(utenteLoggato.getUsername()). Per ogni richiesta restituita, il metodo creaCardRichiesta determina dinamicamente il ruolo dell'utente loggato tramite il flag booleano sonoRichiedente = utenteLoggato.getUsername().equals(req.getRichiedente()):

- **Distinzione visiva dei ruoli:** se l'utente loggato è colui che ha inviato la proposta (sonoRichiedente == true), l'intestazione della card mostra "Richiesta da: Te A: [ricevente]"; se invece ha ricevuto la proposta, mostra "Richiesta da: [richiedente]".
- **Renderizzazione condizionale dei controlli (switch (stato)):**
    - Stato "in sospeso": se l'utente è il **ricevente** (!sonoRichiedente), vengono mostrati sia il pulsante verde "Accetta" (btn-accetta-<id>) sia il pulsante rosso "Rifiuta" (btn-rifiuta-<id>). Se invece l'utente è il **richiedente**, non può auto-accettare la propria proposta, ma visualizza unicamente il pulsante grigio "Annulla" (che invoca comunque rifiutaRichiesta lato server, ma con etichetta e feedback contestualizzati all'annullamento).
    - Stato "rifiutata": mostra l'etichetta statica "Richiesta Annullata" (per il richiedente) o "Richiesta Rifiutata" (per il ricevente).
    - Stato "accettata" o "in attesa di conferma dall'altro utente": abilita il pulsante "Dichiara Terminazione"(btn-termina-<id>) se l'utente corrente non ha ancora confermato, altrimenti mostra il messaggio di cortesia "In attesa della conferma dell'altro utente...".
    - Stato "completato": mostra il bottone "Lascia Recensione" (btn-recensione-<id>) che apre/chiude il form a 5 stelle interattive, oppure la spunta "✓ Recensione inviata" se l'utente ha già espresso il proprio voto.

Skillshare-server

Nel livello backend, la servlet GestoreRichiesteServiceImpl (che estende RemoteServiceServlet) gestisce la logica di business e la persistenza delle richieste sulla collezione HTreeMap<String, RichiestaScambio>denominata "richieste" in MapDB (accessibile tramite getMappaRichieste()):

- **Creazione e validazione (inviaRichiesta):** il server verifica preventivamente che mittente e destinatario siano distinti (if (richiesta.getRichiedente().equals(richiesta.getRicevente())) return false;), proteggendo il database anche da eventuali chiamate RPC anomale. Se l'idRichiesta non è ancora valorizzato, assegna un codice univoco tramite UUID.randomUUID().toString(), inserisce l'oggetto nella mappa "richieste" e consolida la scrittura su disco mediante DatabaseCore.commit().
- **Governo delle transizioni di stato (accettaRichiesta e rifiutaRichiesta):** il server delega le regole di transizione direttamente ai metodi dell'oggetto RichiestaScambio, che applica il pattern **State** tramite req.accetta(nomeutente) e req.rifiuta(nomeutente):
    - In accettaRichiesta, il server verifica che la richiesta esista nel DB e che req.getRicevente().equals(nomeutente): in questo modo, anche qualora un richiedente tentasse di forzare la chiamata RPC, lo stato StatoInSospeso e la servlet bloccherebbero l'operazione restituendo false.
    - In rifiutaRichiesta, lo stato StatoInSospeso consente la transizione a StatoRifiutata sia al ricevente (rifiuto) sia al richiedente (ritiro/annullamento della proposta), bloccandola invece se la richiesta si trova già in uno stato successivo (es. Accettata o Completato, grazie all'implementazione di default false in AbstractStato).

La classe di test GestoreRichiesteServiceImplTest verifica tramite database in memoria (DatabaseCore.enableTestMode()) con 23 test unitari tutti i casi d'uso e i casi limite: corretta creazione nello stato iniziale "In sospeso", blocco dell'auto-scambio senza sporcare il database, filtro delle richieste

ricevute/totali, impossibilità per il richiedente di accettare la propria richiesta, rifiuto/annullamento da ambo le parti, gestione di ID inesistenti e doppia conferma di terminazione.

SkillshareIT

La validazione End-to-End è implementata nelle classi RichiestaScambioIT e GestoreRichiesteIT, supportate dai Page Object BachecaPropostePage e GestisciRichiestePage. I test collaudano mediante Selenium WebDriver l'interazione completa tra due profili distinti:

1. Un primo utente pubblica un annuncio in bacheca ed effettua il logout.
2. Un secondo utente accede alla piattaforma, naviga nella vista "Cerca", localizza l'annuncio nella tabella tramite selettori XPath dinamici e clicca su "Richiedi Scambio". Le asserzioni verificano la comparsa del popup di conferma e la sostituzione del bottone con la dicitura "In attesa di accettazione", confermando il blocco di invii duplicati sullo stesso annuncio.
3. Il primo utente effettua nuovamente il login, accede alla vista GestisciRichiesteGUI e procede all'accettazione o al rifiuto della proposta ricevuta, verificando l'aggiornamento dinamico dei pulsanti di stato. Al termine dell'esecuzione (@AfterEach), gli annunci di test vengono rimossi tramite la procedura di pulizia per preservare l'isolamento del database.

5.6 Utilizza chat interna (UC10)

Il caso d'uso della Chat Interna mette a disposizione dei due utenti coinvolti in uno scambio approvato un canale di messaggistica istantanea privato, persistente e contestualizzato allo specifico annuncio per concordare i dettagli logistici delle lezioni.

Skillshare-shared

Il dominio della messaggistica si fonda sull'entità serializzabile Messaggio, che incapsula quattro attributi:

- idRichiesta: chiave esterna logica che lega in modo univoco la conversazione alla specifica RichiestaScambio.
- mittente: lo username dell'autore del messaggio.
- testo: il contenuto testuale inviato.
- timestamp (java.util.Date): data e ora esatta di invio, utilizzata per garantire l'ordinamento cronologico dello storico.

La comunicazione RPC è definita dall'interfaccia ChatService (mappata sull'endpoint @RemoteServiceRelativePath("chat")) e dalla sua controparte asincrona ChatServiceAsync, che espongono i metodi inviaMessaggio(Messaggio messaggio) e getStoricoMessaggi(String idRichiesta).

Skillshare-client

L'architettura front-end della chat è suddivisa in due componenti GWT distinti per rispettare il principio di responsabilità singola (SRP):

1. **LeMieChatGUI (Vista di riepilogo):** accessibile dalla navbar (nav-btn-chat), interroga il server tramite ottieniTutteLeRichieste e filtra lato client unicamente gli scambi il cui stato risulta "Accettata". Per ciascuno scambio attivo, genera una card (cardChat-lemiechat) che individua automaticamente il nome della controparte (utenteLoggato.equals(richiedente) ? ricevente : richiedente), mostra l'ID dell'annuncio di riferimento e presenta il pulsante "💬 Apri Chat". Cliccando sul pulsante, viene aperta una DialogBox modale contenente il componente ChatPanel e il bottone di chiusura chat-btn-chiudi.
2. **ChatPanel (Componente conversazionale con Polling):** estende Composite e incapsula l'interfaccia di messaggistica vera e propria. Contiene uno ScrollPanel (scrollPanel-chatpanel) che avvolge il VerticalPanel dei messaggi (chat-messaggi-container), affiancato in basso da una TextBox (chat-input-messaggio) e dal bottone d'invio (chat-btn-invia).
    - **Aggiornamento in tempo reale (Polling):** per simulare il comportamento real-time su protocollo HTTP/RPC, ChatPanel istanzia al proprio avvio un com.google.gwt.user.client.Timer che esegue un polling periodico ogni **1500 millisecondi** (scheduleRepeating(1500)), invocando getStoricoMessaggi.Quando l'utente chiude la finestra della chat (chat-btn-chiudi), il timer viene opportunamente arrestato (timer.cancel()) per evitare sprechi di risorse e chiamate di rete inutili in background.
    - **Differenziazione visiva e Auto-scroll:** a ogni ricezione dello storico, i messaggi vengono renderizzati applicando dinamicamente la classe CSS chatmittente-chatpanel (colore blu) se il mittente coincide con l'utente loggato, oppure chatdestinatario-chatpanel (colore verde) se proviene dall'interlocutore. Al termine del rendering, viene invocato scrollToBottom() sullo ScrollPanel per mantenere sempre visibile l'ultimo messaggio ricevuto. Durante l'invio di un nuovo messaggio, il bottone "Invia" viene temporaneamente disabilitato per evitare invii multipli accidentali e riabilitato nella callback.

Skillshare-server

La classe ChatServiceImpl estende RemoteServiceServlet e gestisce la persistenza dei messaggi su MapDB. A differenza delle altre entità memorizzate in mappe chiave-valore (HTreeMap), per i messaggi della chat è stata scelta la struttura dati IndexTreeList<Messaggio>(db.indexTreeList("chat_messaggi", Serializer.JAVA).createOrOpen()), ideale per collezioni sequenziali ad aggiunta frequente (append-only).

- Il metodo inviaMessaggio verifica l'apertura del DB tramite initDb(), accoda il nuovo Messaggio alla IndexTreeList ed esegue DatabaseCore.commit() per garantire la persistenza immediata su disco.
- Il metodo getStoricoMessaggi(String idRichiesta) scansiona la collezione filtrando i soli messaggi aventi idRichiesta corrispondente allo scambio richiesto e, prima di restituire la lista al client, li ordina cronologicamente tramite storico.sort(Comparator.comparing(Messaggio::getTimestamp)).

La logica di backend è validata dalla classe di test unitario ChatBackendTest, che verifica in modalità enableTestMode() il corretto salvataggio di un messaggio, la corrispondenza di testo e mittente e il recupero ordinato dallo storico.

SkillshareIT

La classe ChatIT esegue il test End-to-End completo con Selenium WebDriver: simula la creazione di uno scambio e la sua accettazione per sbloccare il canale di comunicazione, apre la finestra modale della chat con

il primo utente, invia un messaggio tramite #chat-input-messaggio e #chat-btn-invia, effettua l'accesso con il secondo utente e verifica che il messaggio compaia nel #chat-messaggi-container con la formattazione CSS corretta.

5.7 Visualizza profilo utente (UC11)

Il caso d'uso "Visualizza Profilo Utente" permette sia agli iscritti autenticati sia ai visitatori ospiti (dalla Home Page) di consultare la scheda pubblica di qualsiasi membro della community, ispezionandone i dati anagrafici, la biografia, i tag di competenza, il livello di esperienza, i badge di merito, la media delle recensioni e lo storico completo degli scambi effettuati.

Skillshare-shared

Il modulo condiviso si appoggia a tre DTO serializzabili (java.io.Serializable) per trasferire al client tutte le informazioni che compongono la scheda utente:

- Utente: incapsula le informazioni anagrafiche e di presentazione (nome, cognome, username, email, dataNascita, bio, fotoProfilo, tagCompetenze).
- StatisticheUtente: trasporta i dati di reputazione e gamification associati allo username (scambiCompletati, mediaRecensioni, numeroRecensioni, livello di esperienza e la lista dei tag/badge di merito conseguiti).
- RichiestaScambio: impiegata per popolare il riepilogo tabellare dello storico degli scambi dell'utente.

Le interfacce RPC GestoreProfilo (@RemoteServiceRelativePath("profilo")) e GestoreProfiloAsync espongono i metodi per recuperare il profilo pubblico (ottieniProfiloUtente / recuperaProfilo) e le metriche di reputazione (ottieniStatisticheUtente),
mentre GestoreRichiesteService e GestoreRichiesteServiceAsync forniscono ottieniTutteLeRichieste(String nomeutente) per estrarre lo storico delle sessioni.

Skillshare-client

L'interfaccia grafica è gestita dalla classe ProfiloUtenteGUI, che costruisce una finestra modale GWT (DialogBox / PopupPanel) richiamabile trasversalmente da più sezioni dell'applicazione:

- **Accesso personale:** cliccando sulla voce "Profilo" nella barra di navigazione (AppNavbar) oppure sul pulsante "Visualizza il mio profilo pubblico" all'interno di PersonalizzaProfiloGui.
- **Accesso pubblico (anche per utenti non loggati):** cliccando sul nome dell'autore di un annuncio (link-profilo-<username>) all'interno della BachecaProposteGUI oppure nella sezione "Ultimi Annunci Pubblici" della HomePageGUI.

All'apertura del popup, ProfiloUtenteGUI invia le chiamate RPC asincrone al server e renderizza dinamicamente la scheda (stilizzata tramite personalizzaProfilo.css e items-comuni.css):

1. **Dati anagrafici e competenze:** mostra l'immagine del profilo circolare (o un avatar di default se non specificata), Nome e Cognome, Username, Data di Nascita, Email, Biografia testuale e i Tag delle competenze offerte.

2. **Reputazione, Livello e Badge di Merito:** espone in chiaro il rango raggiunto dall'utente (es. *Novizio*, *Esperto*, *Maestro*), il contatore Scambi completati, la Media recensioni su 5 e i badge grafici di merito (es. *Novizio*, *Eccellente*).
3. **Pulsante contestuale "Modifica Profilo":** la GUI verifica se il profilo ispezionato appartiene all'utente attualmente loggato; solo in caso affermativo affianca al bottone "Chiudi" il pulsante "✏️ Modifica Profilo", che reindirizza alla vista PersonalizzaProfiloGui.
4. **Tabella dello Storico Scambi:** cliccando sul pulsante "Visualizza Storico Scambi", viene invocata la chiamata asincrona ottieniTutteLeRichieste per l'utente visualizzato. Il client popola dinamicamente la tabella #tabella-storico (mostrando per ogni scambio *ID Annuncio, Richiedente, Ricevente* e *Stato*), oppure visualizza l'etichetta #msg-storico-vuoto qualora l'utente non abbia ancora scambi registrati.

Skillshare-server

Sul backend, le richieste vengono gestite dalla servlet GestoreProfiloImpl, che si avvale della catena GestoreProfiloProxy → GestoreProfiloReale e di GestoreLeveling:

- **Lookup in $O(1)$ e Sicurezza dei Dati Sensibili (Pattern Proxy / Sanificazione DTO):** per recuperare il profilo a partire dallo username o dalla mail, GestoreProfiloReale interroga DatabaseUtente (sfruttando la mappa indice_username in tempo costante $O(1)$). Prima di restituire l'oggetto Utente al client, il server istanzia un nuovo oggetto Utente copiando esclusivamente gli attributi pubblici (inclusi dataNascita, bio, foto e tag) e **omettendo la password (password = null)**. Questo garantisce che le credenziali riservate non escano mai dal perimetro del server.
- **Arricchimento delle Statistiche (Pattern Decorator):** per fornire i dati di reputazione, il server richiama GestoreLeveling.getStatistiche(username), che estrae l'oggetto StatisticheUtente dalla mappa MapDB "statistiche" e lo passa attraverso la catena di Decorator (DescrizioneUtenteBase, NovizioDecorator, EspertoDecorator, MaestroDecorator, DecoratoreRecensioni) per calcolare dinamicamente livello e badge.

I test unitari nella classe GestoreProfiloTest verificano la correttezza dei dati restituiti dal recupero profilo e certificano tramite assertNull(profilo.getPassword()) che la password venga sempre oscurata.

SkillshareIT

I test di accettazione Selenium sono distribuiti in due classi:

- HomeProfiloIT (con il Page Object HomePage): verifica che un visitatore non autenticato possa cliccare sullo username di un autore nella sezione "Ultimi Annunci Pubblici" della Home Page e ispezionare correttamente il popup del profilo pubblico.
- VisualizzaProfiloIT (con il Page Object ProfiloUtentePage): autentica un utente, apre la scheda profilo, verifica la presenza di biografia, tag, statistiche di gamification e collauda il pulsante "Visualizza Storico Scambi", asserendo la corretta renderizzazione della tabella #tabella-storico (o del messaggio #msg-storico-vuoto).

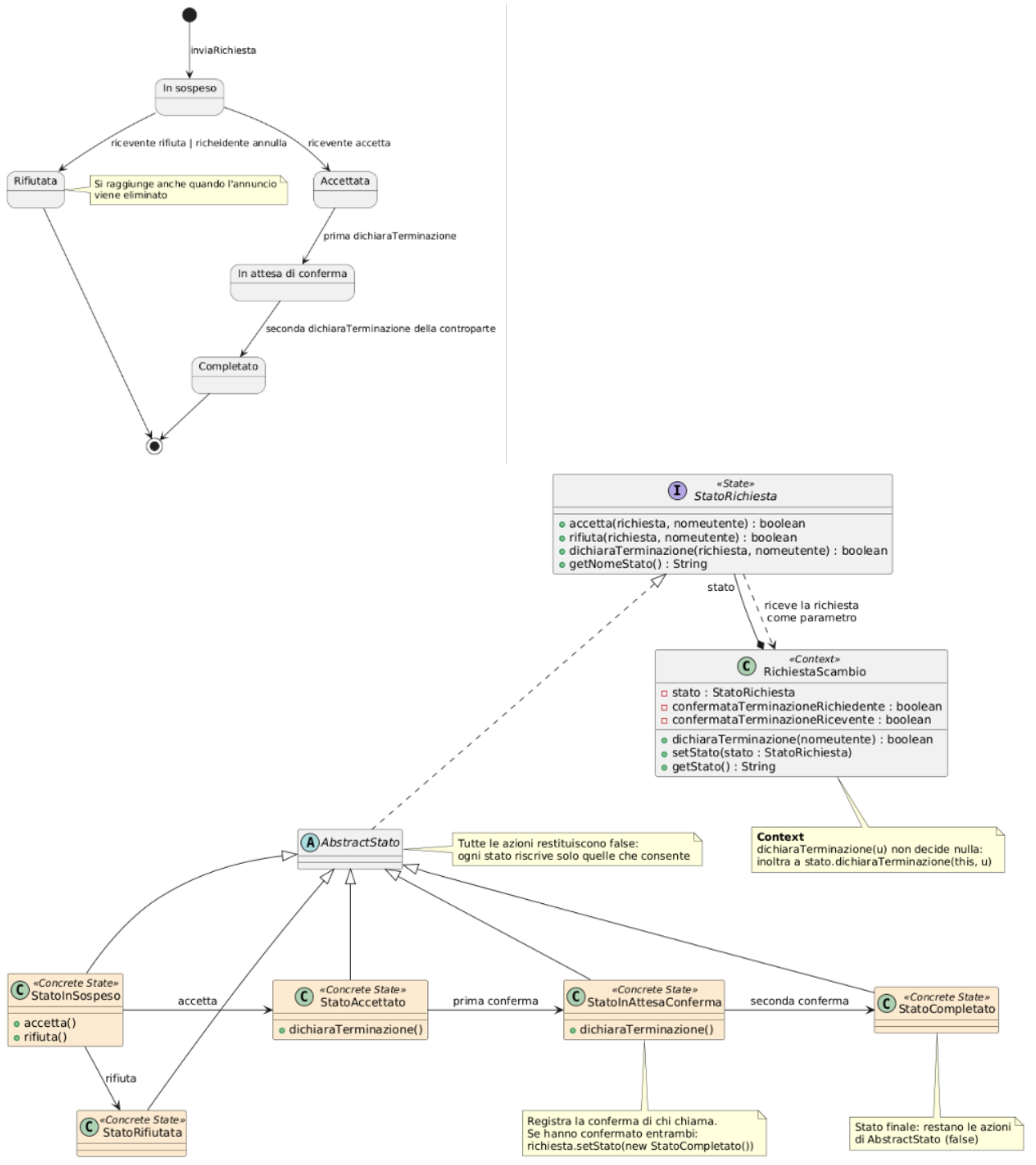
5.8 Chiusura sessione e feedback (UC12 e UC13)

I casi d'uso "Conferma completamento sessione" (UC12) e "Inserisci Recensione" (UC13) regolano la fase conclusiva dello scambio didattico: una richiesta accettata si chiude ufficialmente solo quando entrambi i partecipanti dichiarano la terminazione della lezione (protocollo a doppia conferma). A quel punto viene sbloccata la possibilità per ciascun utente di valutare la controparte con un punteggio da 1 a 5 stelle e un commento testuale, innescando l'aggiornamento automatico delle statistiche, dei livelli di esperienza e dei badge di merito (Gamification).

Skillshare-shared

Il livello shared definisce i modelli dati e i contratti per i pattern **State** e **Observer**:

- **Gestione dello Stato (StatoRichiesta e AbstractStato):** l'interfaccia StatoRichiesta espone le azioni consentite su una richiesta (accetta, rifiuta, dichiaraTerminazione, getNomeStato), mentre la classe astratta AbstractStato restituisce false di default per tutte le operazioni non permesse, lasciando agli stati concreti (StatoInSospeso, StatoAccettato, StatoInAttesaConferma, StatoCompletato, StatoRifiutata) il compito di sovrascrivere solo le transizioni valide.
- **Entità per Valutazioni e Gamification:**
    - Recensione: oggetto serializzabile che incapsula la valutazione numerica (da 1 a 5) e il commento testuale.
    - StatisticheUtente: memorizza per ogni username il numero di scambiCompletati, la mediaRecensioni, il livello raggiunto (*Novizio*, *Esperto*, *Maestro*) e i tag/badge di qualità (*Ben Valutato*, *Eccellente*, *Leggendario*).
    - OsservatoreScambio: interfaccia Observer che definisce i metodi reattivi DopoScambioCompletato(RichiestaScambio richiesta) e DopoRecensioneAggiunta(String usernameRecensito, int valutazione).



Skillshare-client

L'interazione lato utente avviene nella vista GestisciRichiesteGUI (e i risultati sono visibili in ProfiloUtenteGUI):

- **Doppia conferma di terminazione (UC12):** quando uno scambio si trova in StatoAccettato, la card mostra il pulsante "Dichiara Terminazione" (btn-termina-<id>). Al click del primo utente, parte la chiamata RPC dichiaraTerminazione(idRichiesta, username): lo stato passa a StatoInAttesaConferma e per l'utente che ha già confermato il bottone viene sostituito dal messaggio "In attesa della conferma

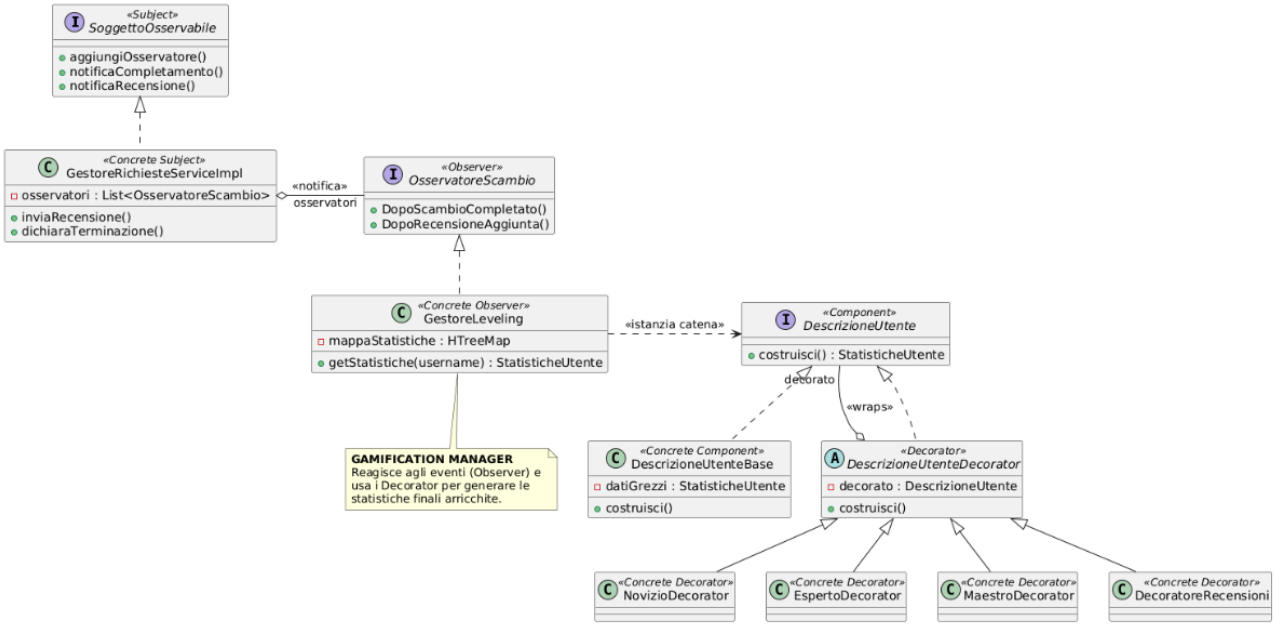
dell'altro utente...", impedendogli di chiudere unilateralmente lo scambio. Quando anche la controparte clicca su "Dichiara Terminazione", la richiesta passa a StatoCompletato.

- **Inserimento Recensione (UC13):** solo al raggiungimento dello stato "Completato", sulla card compare il pulsante "Lascia Recensione" (btn-recensione-<id>). Cliccandolo si espande il pannello .formRecensione-richiesteRicevute, dotato di un selettore interattivo a 5 stelle cliccabili (.stellaStyle-richiesteRicevute) e di una TextArea per il commento opzionale. Il client blocca l'invio se non è stata selezionata alcuna stella; in caso di chiusura/annullamento del form, i campi vengono resettati senza alterare i dati. Una volta inviata la valutazione tramite inviaRecensione, il pulsante viene nascosto e sostituito dalla dicitura "✓ Recensione inviata", impedendo recensioni multiple.

Skillshare-server

Sul backend, il coordinamento tra la chiusura dello scambio e la gamification è gestito da GestoreRichiesteServiceImpl e GestoreLeveling attraverso tre design pattern (**State**, **Observer** e **Decorator**):

1. **Pattern State per la Doppia Conferma (RichiestaScambio):** Nelle prime versioni lo stato era gestito tramite una semplice variabile String e una serie di if/else. Con l'adozione del pattern **State**, RichiestaScambio (Context) mantiene i flag confermataTerminazioneRichiedente e confermataTerminazioneRicevente e delega la chiamata dichiaraTerminazione(nomeutente) allo stato corrente:
    - StatoAccettato registra la prima conferma dell'utente chiamante e transiziona la richiesta in StatoInAttesaConferma.
    - StatoInAttesaConferma verifica il ruolo di chi invoca il metodo: solo quando entrambi i flag (richiedente e ricevente) risultano true, effettua la transizione finale a new StatoCompletato(). In StatoCompletato tutte le azioni ereditano il comportamento inattivo (false) di AbstractStato.
2. **Pattern Observer per disaccoppiare Richieste e Gamification (SoggettoOsservabile / OsservatoreScambio):**Per evitare che GestoreRichiesteServiceImpl conoscesse i dettagli implementativi della gamification (rispettando **SRP** e **OCP**), la servlet implementa l'interfaccia SoggettoOsservabile e registra nel costruttore un'istanza di GestoreLeveling (Concrete Observer) tramite aggiungiOsservatore:
    - Quando dichiaraTerminazione porta lo stato a "Completato", la servlet invoca notificaCompletamento(richiesta): GestoreLeveling.DopoScambioCompletato incrementa di 1 il contatore degli scambi completati per entrambi gli utenti coinvolti su MapDB.
    - Quando inviaRecensione riceve un voto valido (dopo aver verificato lato backend che lo stato sia "Completato", che l'utente faccia parte dello scambio e che !richiesta.isRecensioneInviata(recensore) per bloccare recensioni duplicate), salva la recensione ed esegue notificaRecensione(usernameRecensito, valutazione): GestoreLeveling.DopoRecensioneAggiunta ricalcola la media aritmetica delle valutazioni dell'utente recensito.




```java
//@Override di SoggettoOsservabile
@Override  1 usage  samuesamue
public void aggiungiOsservatore(OsservatoreScambio o) {
    osservatori.add(o);
}

@Override  1 usage  samuesamue
public void notificaCompletamento(RichiestaScambio richiesta) {
    for (OsservatoreScambio o : osservatori) {
        o.DopoScambioCompletato(richiesta);
    }
}

@Override  1 usage  samuesamue
public void notificaRecensione(String usernameRecensito, int valutazione) {
    for (OsservatoreScambio o : osservatori) {
        o.DopoRecensioneAggiunta(usernameRecensito, valutazione);
    }
}
```

3. **Pattern Decorator per Livelli e Badge (DescrizioneUtente):** La classe GestoreLeveling utilizza una catena di **Decorator** per arricchire dinamicamente l'oggetto StatisticheUtente con livelli e badge senza usare blocchi condizionali monolitici:
    - DescrizioneUtente (Component) e DescrizioneUtenteBase (Concrete Component): avvolgono i dati grezzi estratti dalla mappa "statistiche" di MapDB creando un oggetto pulito con livello e tag inizialmente vuoti.
    - DescrizioneUtenteDecorator (Decorator astratto): mantiene il riferimento al componente decorato e delega la costruzione alle sottoclassi.
    - NovizioDecorator, EspertoDecorator, MaestroDecorator (Concrete Decorators per l'esperienza): valutano in sequenza il numero di scambi completati, assegnando e sovrascrivendo progressivamente il livello e il relativo badge al superamento delle soglie previste.

- DecoratoreRecensioni (Concrete Decorator trasversale per la qualità): opera in modo indipendente sul numero di recensioni ricevute e sulla media voti, assegnando i badge di merito qualitativi (*Ben Valutato*, *Eccellente*, *Leggendario*).

SkillshareIT

- **Test Unitari (GestoreRichiesteServiceImplTest):** testano con database in memoria le transizioni del pattern State (impossibilità di terminare uno scambio non accettato o con una sola conferma), il blocco delle recensioni duplicate e l'aggiornamento automatico di StatisticheUtente tramite la catena Observer + Decorator.
- **Test di Accettazione Selenium (RecensioneIT e GestoreRichiesteIT):** simulano tramite i Page Object GestisciRichiestePage e ProfiloUtentePage l'intero flusso E2E tra due utenti: invio e accettazione della richiesta, doppia conferma di terminazione da ambo le parti fino a StatoCompletato, sblocco del form recensione, selezione delle 5 stelle con invio del commento, scomparsa del bottone per prevenire un secondo invio e verifica finale sul profilo pubblico dell'effettivo incremento degli scambi completati, della media recensioni e della comparsa dei badge (Novizio, Eccellente).

5.9 Creazione della wishlist (UC14)

Il modulo relativo alla gestione della Wishlist permette agli utenti di inserire e salvare i servizi che stanno cercando all'interno della piattaforma.

Skillshare-shared

L'entità di riferimento del modulo è la classe *ServizioRichiesto*. Questa classe modella il desiderio dell'utente incapsulando tre informazioni chiave: il nome del servizio, la categoria di competenza associata e una breve descrizione. Per garantire che il framework GWT possa trasmettere correttamente gli oggetti instanziati tra il client e il server, la classe implementa l'interfaccia *java.io.Serializable*.

L'interfaccia *WishlistService* estende *RemoteService* e definisce l'endpoint per il server tramite l'annotazione *@RemoteServiceRelativePath("wishlist")*, indicando l'URL esatto a cui il frontend inoltrerà le richieste. Essa espone i metodi per l'aggiunta di un nuovo elemento (*aggiungiServizio*) e per il recupero dell'intera lista (*getWishlistUtente*). Entrambi i metodi utilizzano la *mailUtente* come identificatore per mappare i dati all'account corretto.

Per il lato client, viene fornita l'interfaccia *WishlistServiceAsync*. Questa replica le firme dei metodi del servizio aggiungendo un parametro *AsyncCallback*, essenziale per eseguire le chiamate di rete in modo asincrono, evitando blocchi dell'interfaccia grafica in attesa della risposta del server.

Skillshare-client

Le interfacce grafiche relative alla gestione della wishlist si articolano in due viste principali, creazione e consultazione.

La classe *CreaWishlistGUI* costituisce il modulo di inserimento. L'interfaccia si assicura che l'utente compili ogni campo necessario (servizio richiesto, tag competenza e descrizione) validandoli localmente. In caso di fallimento della validazione, un feedback visivo istantaneo blocca l'operazione. Se la validazione ha successo, il client instanzia un oggetto *ServizioRichiesto* e avvia una chiamata asincrona tramite

WishlistServiceAsync.aggiungiServizio. Una AsyncCallback gestisce l'esito: in caso positivo (*onSuccess*), il modulo viene pulito per consentire un nuovo inserimento, altrimenti notifica l'eccezione lanciata dal server (*onFailure*).

La classe *MiaWishlistGUI* permette la consultazione dei servizi desiderati. Essa lancia una richiesta asincrona tramite *getWishlistUtente* all'apertura della vista. Una volta ricevuta l'*ArrayList<ServizioRichiesto>*, il client itera sui risultati per generare dinamicamente le *card* descrittive di ogni servizio.

L'interfaccia client funge da ponte verso la logica di matchmaking. Durante il rendering della lista, subito al di sotto di ciascuna *card* personale, il codice istanzia dinamicamente un componente aggiuntivo (*ProfiliSuggeritiWidget*) passando la mail dell'utente e il *tagCompetenza*.

Skillshare-server

Il livello server del modulo Wishlist è racchiuso nella classe *WishlistServiceImpl*, che estende *RemoteServiceServlet*. Questo componente si occupa di validare le richieste ricevute dal client, di gestire le eccezioni e di persistere i dati in modo permanente all'interno del database MapDB.

L'accesso al database viene gestito nel metodo *initDb()*, il quale si assicura che la connessione al DatabaseCore sia aperta. I dati si trovano in un HTreeMap denominata *wishlist_servizi*. Questa mappa utilizza la mail dell'utente come chiave univoca e associa a essa un'intera *ArrayList<ServizioRichiesto>*. Il metodo *aggiungiServizio* recupera la lista preesistente (se presente) e vi aggiunge in coda il nuovo servizio, confermando l'operazione su disco tramite *DatabaseCore.commit()*.

Nel metodo *getWishlistUtente*, se la mail non viene trovata nel database (ovvero quando l'utente interroga la propria wishlist per la prima volta), il server restituisce una lista vuota. In parallelo, il server valida gli input, sollevando eccezioni se la mail passata dal client risulta assente o vuota.

Il comportamento del server è certificato da test di unità implementati in *WishlistServiceImplTest* e *ServizioRichiestoTest*. I test di unità simulano l'avvio della servlet con un database virtuale (*DatabaseCore.enableTestMode()*). Si verificano percorsi critici come il corretto accumulo di più servizi sulla stessa chiave utente, l'isolamento dei dati tra utenti diversi (test testAggiungiServizio_UtentiDiversiSeparati) e la robustezza del sistema di fronte a input anomali (test sulle eccezioni in caso di mail nulle). Infine, si accerta che la serializzazione e la modifica dell'entità ServizioRichiesto rispettino i contratti del framework (costruttore vuoto, getter e setter validi).

SkillshareIT

Dato che la Wishlist è strettamente legata all'identità dell'utente, la classe di test *WishlistIT* implementa una fase di setup. Prima di ogni singolo test (*@BeforeEach*), viene generato un utente univoco che viene registrato e loggato automaticamente utilizzando il Page Object *RegistrazionePage*. In questo modo, ogni scenario parte da uno stato pulito e isolato, simulando le precondizioni del caso d'uso. Al termine (*@AfterEach*), il sistema forza il logout.

I test si affidano alla classe *WishlistPage*. Questo oggetto espone metodi ad alto livello come *apriDallaNavbar()*, *vaiAlFormAggiunta()*, *compila()* e *aggiungi()*, incapsulando al suo interno le logiche di sincronizzazione e di ricerca degli elementi tramite i loro ID. Questa separazione rende i test leggibili e focalizzati esclusivamente sulla logica di business.

Il test *testWishlistVuotaPerUtenteNuovo* naviga verso la vista della wishlist appena dopo la registrazione, accertandosi tramite il metodo *isWishlistVuota()* che il server restituisca correttamente una schermata vuota.

Nello scenario *testAggiuntaServizio*, il test compila il form con dati validi, ne simula l'invio e verifica la comparsa del messaggio di successo. Successivamente, ricarica la pagina della lista interrogando il server e verifica che la nuova card HTML sia stata generata dinamicamente con i dati attesi.

Il test *testCampiObbligatoriMancanti* simula un utente distratto che compila solo il nome del servizio omettendo i campi restanti. Il test verifica che il sistema blocchi l'invio segnalando l'errore a schermo ("*Compila tutti i campi*"). Quando i dati vengono successivamente integrati dal metodo *completaCampiMancanti()*, l'inserimento procede con successo.

5.10 Matchmaking (UC15)

Il modulo di Matchmaking preleva i tag e i servizi precedentemente memorizzati dagli utenti all'interno della propria Wishlist e li incrocia con le competenze offerte dagli altri profili presenti nel database, generando un flusso di utenti compatibili pronti per lo scambio.

Skillshare-shared

L'interfaccia *MatchmakingService* estende *RemoteService* e mappa l'endpoint di backend tramite l'annotazione *@RemoteServiceRelativePath("matchmaking")*. Espone le firme per due ambiti funzionali distinti. Il primo è la ricerca attiva, operata dal metodo *trovaProfiliCompatibili*, che riceve la mail dell'utente richiedente e il *tagRicercato* per restituire una lista mirata di oggetti *Utente*. Il secondo ambito riguarda la persistenza delle interazioni sociali, gestita dai metodi *salvaProfilo*, *rimuoviProfilo* e *getProfiliSalvati*, i quali permettono di recuperare le preferenze (i “cuori”) espresse dall'utente verso altri membri della piattaforma.

L’interfaccia *MatchmakingServiceAsync*. Questa mappa le operazioni del servizio originale, iniettando in ogni firma il parametro *AsyncCallback*. Questa struttura delega la gestione della richiesta a chiamate di rete non bloccanti, assicurando che l'interfaccia utente rimanga completamente reattiva in attesa che il server completi l'algoritmo di corrispondenza o l'aggiornamento del database.

Skillshare-client

Il livello client per il matchmaking non è implementato come una pagina a sé stante, bensì come un componente modulare denominato *ProfiliSuggeritiWidget*. Questo widget è progettato per essere iniettato dinamicamente sotto ogni elemento della Wishlist, arricchendo l'esperienza utente contestualmente al servizio desiderato.

All'inizializzazione, il widget invoca *trovaProfiliCompatibili* fornendo la competenza target (*tagRicercato*). Se il server restituisce una lista valida di utenti compatibili, il client non procede immediatamente al rendering, ma effettua una seconda chiamata (*getProfiliSalvati*) per recuperare lo stato storico delle preferenze (i profili a cui l'utente loggato ha già messo "cuore" in precedenza). Solo al termine di questo flusso asincrono, i dati combinati vengono passati al metodo *mostraCard*.

Ogni *card* generata include un pulsante interattivo delegato all'espressione delle preferenze. Il metodo *gestisciToggleProfilo* agisce come una macchina a stati per questo bottone: ne legge lo stato visivo attuale (cuore bianco o rosso) e inoltra dinamicamente la chiamata RPC corretta (*salvaProfilo* o *rimuoviProfilo*). Per prevenire *race conditions* o click compulsivi prima della risposta del server, il pulsante viene disabilitato temporaneamente all'inizio dell'operazione (*bottone.setEnabled(false)*) e riabilitato unicamente all'interno della *AsyncCallback*.

Skillshare-server

La classe *MatchmakingServiceImpl* implementa l'interfaccia RPC e incapsula la logica di ricerca e persistenza, interagendo strettamente con il database MapDB.

L'algoritmo di corrispondenza esegue una scansione completa della tabella utenti. Per garantire la massima precisione, il server normalizza sia i tag inseriti dall'utente nella wishlist, sia i tag associati ai profili scansionati. L'incrocio avviene tramite il metodo *haCompetenza*, che garantisce una corrispondenza esatta dei termini. Inoltre, un controllo logico preliminare assicura che l'utente che effettua la ricerca venga automaticamente escluso dai propri risultati.

Le interazioni (il salvataggio o la rimozione dei "cuori") sono gestite tramite una HTreeMap dedicata, *preferenze_match*, che memorizza le relazioni sotto forma di *ArrayList<String>* associate alla mail dell'utente richiedente. Per evitare ridondanze, metodi come *salvaProfilo* verificano preventivamente che l'utente non sia già presente nella lista (*!salvati.contains(...)*), per poi rendere permanenti le modifiche sul database invocando *DatabaseCore.commit()*.

Il collaudo della classe è garantito dai test della classe *MatchmakingServiceImplTest*. Tramite *DatabaseCore.enableTestMode()*, viene generato un database temporaneo in memoria, precaricato con dati fittizi. I test verificano l'efficacia dell'algoritmo di ricerca, l'assenza di falsi positivi in caso di competenze non registrate, e prevengono casi limite come l'auto-salvataggio del proprio profilo, che il server deve intercettare bloccando l'operazione con un'eccezione dedicata. Inoltre, le asserzioni di sicurezza accertano tramite *assertNull(match.getPassword())* che le credenziali vengano effettivamente epurate prima della trasmissione.

SkillshareIT

La validazione end-to-end del motore di Matchmaking è affidata a una suite di collaudo basata su Selenium WebDriver, progettata per simulare l'intero flusso utente, dall'espressione di un desiderio fino all'interazione attiva con le *card* dei profili suggeriti. Tutti i test si trovano nella classe *MatchmakingIT*.

Per assicurare che ogni scenario operi in un contesto deterministico, la configurazione *@BeforeEach* crea e autentica dinamicamente un nuovo profilo utente, generando un indirizzo e-mail univoco. Questa procedura si appoggia a *RegistrazionePage* e garantisce che il test operi sempre su una cronologia vuota, prevenendo falsi positivi derivanti da esecuzioni precedenti.

L'interazione con i nodi HTML generati da GWT è incapsulata nella classe *MatchmakingPage*. Essa espone metodi di navigazione e validazione ad alto livello (come *attendiSuggerimentiVisibili* e *cliccaPrimoBottoneProfilo*), mascherando la complessità dei selettori.

Lo scenario di collaudo *testToggleSalvataggioERimozioneProfilo* verifica l'integrazione tra la Wishlist e il Matchmaking. Il test automatizza l'aggiunta di un "Corso Java" nei desideri, forzando la comparsa del widget dei suggerimenti. Successivamente, si adatta allo stato del database: se vengono renderizzati profili compatibili, il test simula due click consecutivi sulla stessa *card*, asserendo che l'interfaccia rifletta le transizioni di stato (passando da "Salva Profilo" a "Salvato", e viceversa). Qualora non vi siano match disponibili, asserisce che il sistema gestisca l'assenza di dati stampando a schermo il messaggio esatto "Nessun profilo trovato al momento".